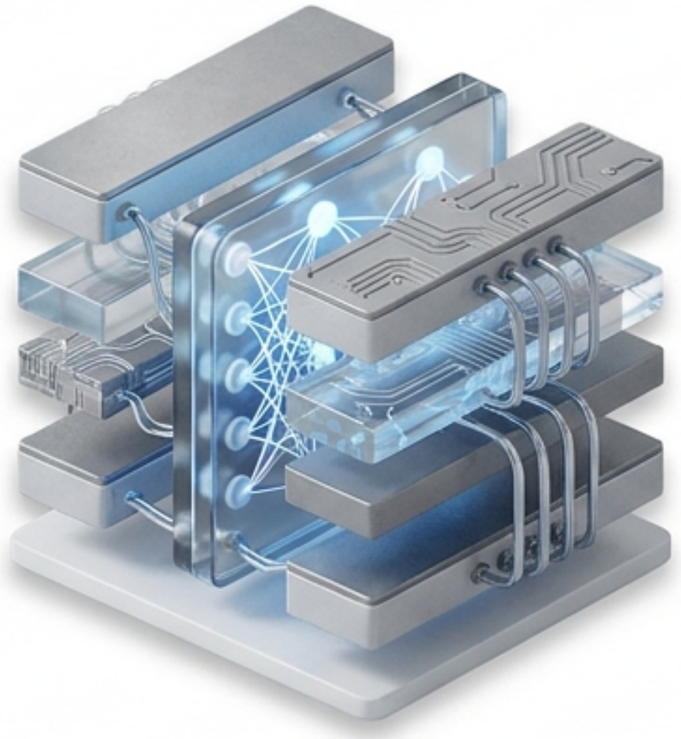
Xây dựng Phần đầu YOLO (Prediction Head)

Phần đặc trưng xuất ra từ ResNet sẽ được bóp phẳng và đưa qua 2 lớp Dense để biến thành thông tin Grid tọa độ.



```python
from keras import layers

grid_size = 6
num_labels = 91

inputs = keras.Input(shape =( image_size , image_size , 3))
x = backbone(inputs)
# Makes our backbone outputs smaller and then flattens the output
x = layers.Conv2D(512, (3, 3), strides =(2, 2))(x)
x = layers.Flatten()(x)

# Passes our flattened feature maps through densely connected layers
x = layers.Dense(2048, activation="relu", kernel_initializer="glorot_normal")(x)
x = layers.Dropout(0.5)(x)
x = layers.Dense(grid_size * grid_size * (num_labels + 5))(x)
```

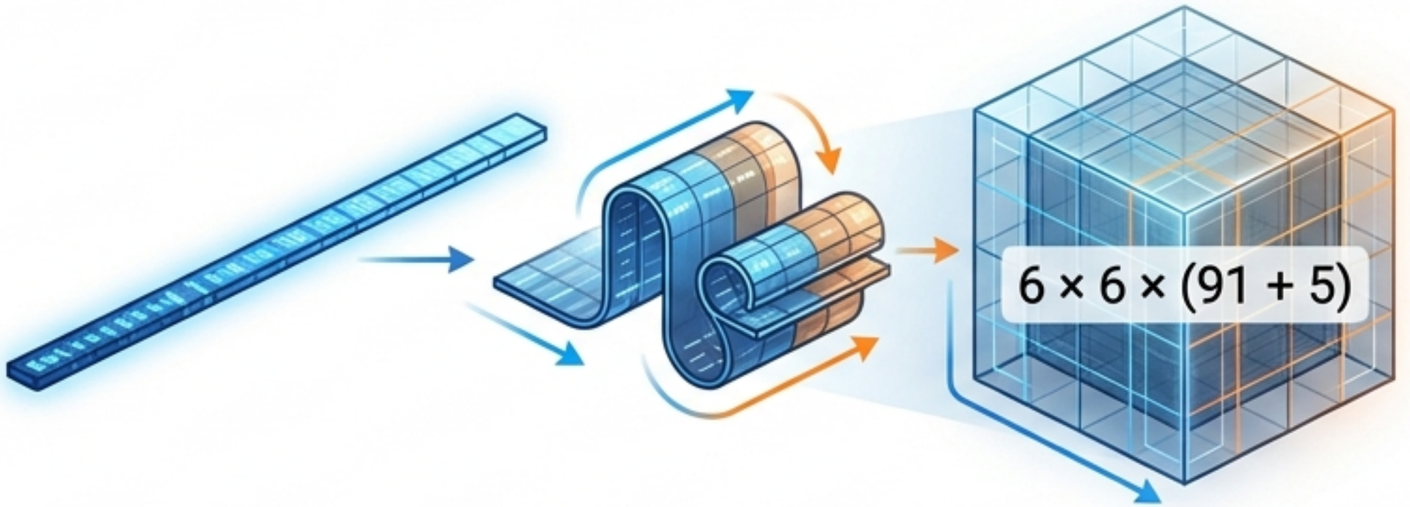
Bóp phẳng từ ResNet

Dense layers biến thành thông tin Grid

Gemini Notebook

Định hình Tensor Đầu ra

Chuyển vector 1 chiều cuối cùng thành dạng Tensor 6× 6× (91+ 5). Sau đó cắt tách Box và Class riêng biệt.




```python
# Reshapes outputs to a 6 x 6 grid
x = layers.Reshape ((grid_size , grid_size , num_labels + 5))(x)

# Split box and class predictions
# The first 5 values: x, y, w, h, confidence
box_predictions = x[..., :5]

# The next 91 values: Class probabilities using Softmax
class_predictions = layers.Activation("softmax")(x[..., 5:])

outputs = {"box": box_predictions , "class": class_predictions}
model = keras.Model(inputs , outputs)
```

5 giá trị đầu:
x, y, w, h, confidence

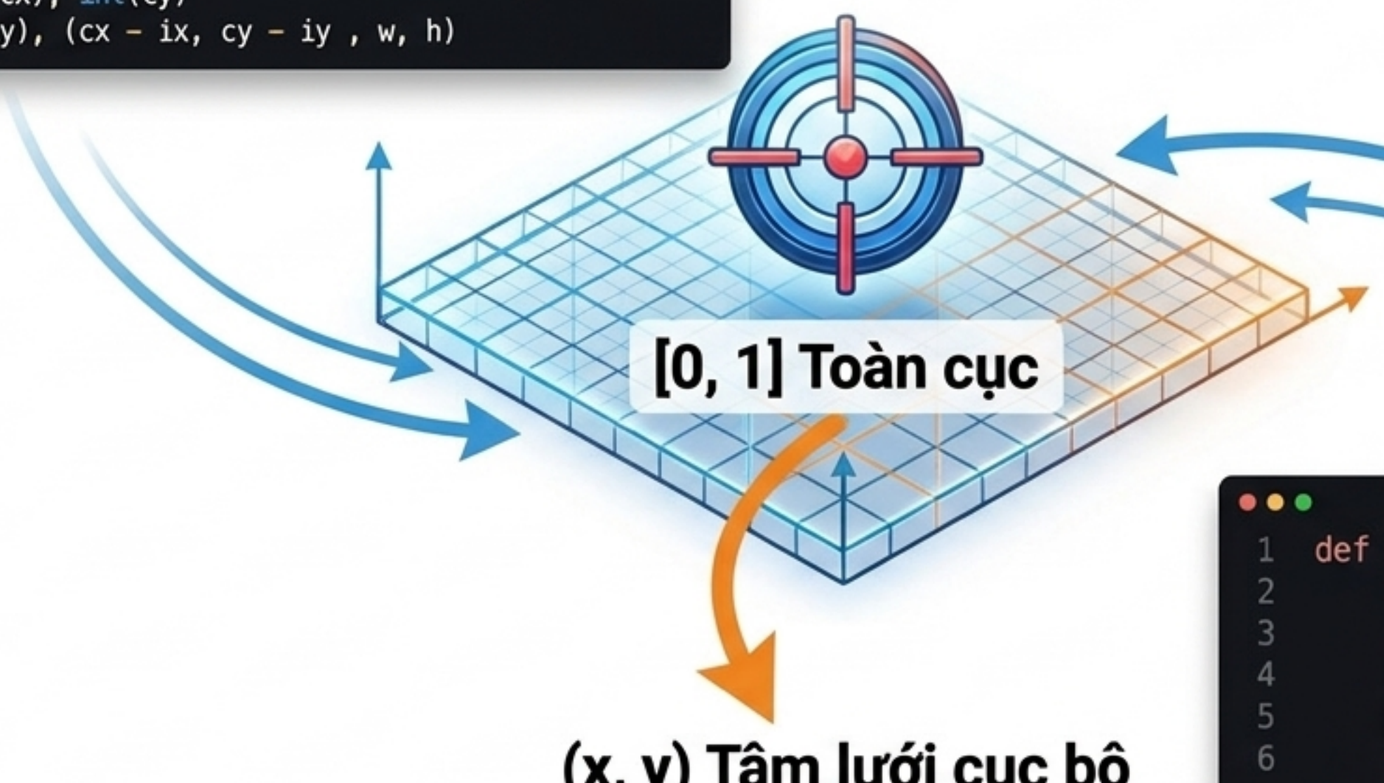
91 giá trị sau:
Xác suất Class (Softmax)

Gemini Notebook

Ánh xạ Tọa độ Hộp vào Lưới Grid

Dữ liệu nhãn đang là kích thước toàn cục ([0, 1]). Cần chuyển hệ tọa độ tâm (x, y) vào từng ô lưới nhỏ tương ứng.

```python
def to_grid(box):
    x, y, w, h = box
    cx, cy = (x + w / 2) * grid_size , (y + h / 2) * grid_size
    ix, iy = int(cx), int(cy)
    return (ix, iy), (cx - ix, cy - iy , w, h)
```




```python
def from_grid(loc , box):
    (xi, yi), (x, y, w, h) = loc , box

    x = (xi + x) / grid_size - w / 2
    y = (yi + y) / grid_size - h / 2
    return (x, y, w, h)
```

Gemini Notebook

Sinh Dữ liệu Lưới (Grid Targets)



Vòng lặp ánh xạ hộp vào lưới. Mỗi ô lưới sẽ chứa thông tin về lớp đối tượng và hộp giới hạn.

```python
import numpy as np
import math

class_array = np.zeros ((len(metadata), grid_size , grid_size))
box_array = np.zeros((len(metadata), grid_size , grid_size , 5))

for index , sample in enumerate(metadata):
    boxes , labels = sample["boxes"], sample["labels"]
    for box , label in zip(boxes , labels):
        # Transforms the box to the grid coordinate system
        (xi , yi), (grid box) = to grid(box)
        box_array[index , yi , xi] = [*grid_box , 1.0] # 1.0 is confidence

        # Makes sure the class label for the box's center location matches
        class_array[index , yi, xi] = label
```

(Chỉ gán nếu ô đó chứa vật)

Gemini Notebook

Toán học: Cấu trúc Vector Đầu ra

Kết quả của đoạn Code vừa rồi tạo ra Vector gồm:



Hình 12.7: Vector mục tiêu huấn luyện

- **pc** : Confidence Score (Xác suất có chứa vật thể). Bằng 1.0 nếu có vật.
- **bx, by** : Tọa độ tâm của hộp giới hạn (Theo ô lưới).
- **bw, bh :** Chiều rộng, chiều cao của hộp (Toàn cục).

Bounding Box Data

- **c1, c2, ...ck** : Xác suất thuộc về class i (Vd: Chó, Mèo, Xe).

Gemini Notebook

Sử dụng tf.data thiết lập Pipeline



Tải ảnh song song (8 tiến trình) → Ghép nối Tensor → Chia Batch (16) → Prefetch

```python
import tensorflow as tf

def load_image(path):
    x = tf.io.read_file(path)
    x = tf.image.decode_jpeg(x, channels =3)
    return preprocessor(x)

images = tf.data.Dataset.from_tensor_slices ([x["path"] for x in metadata ])
images = images.map(load_image , num_parallel_calls =8)

labels = {"box": box_array , "class": class_array}
labels = tf.data.Dataset.from_tensor_slices(labels)

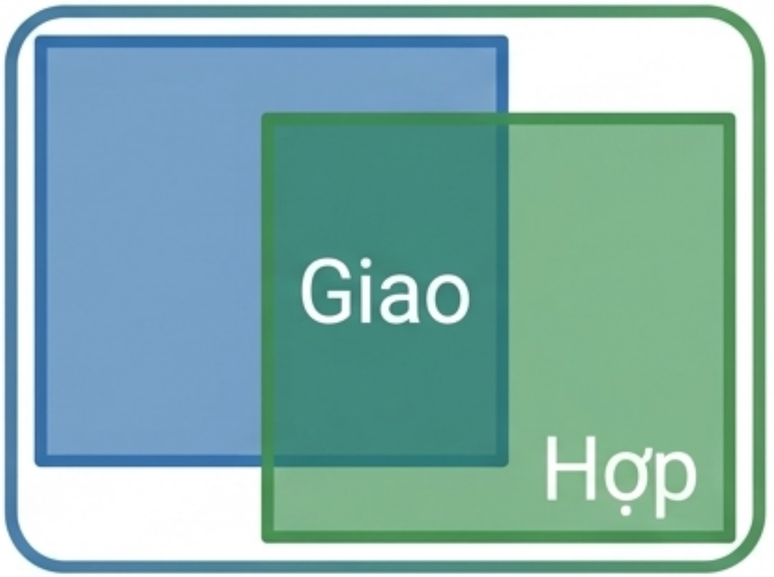
# Creates a merged dataset and batches it
dataset = tf.data.Dataset.zip(images , labels).batch (16).prefetch (2)
val_dataset , train_dataset = dataset.take (500) , dataset.skip (500)
```

Lệnh '**prefetch**' để chống thắt cổ chai GPU

Gemini Notebook

Công cụ 1: Intersection over Union (IoU)

IoU là công thức toán học cốt lõi để tính toán độ trùng khớp giữa 2 Hộp giới hạn. Dao động từ 0 (Không chạm nhau) đến 1 (Trùng khớp hoàn toàn).



IoU = Diện tích phần Giao nhau / Diện tích phần Hợp nhau

$$IoU = Area(Box1 \cap Box2) \,/\, Area(Box1 \cup Box2)$$

Khác biệt: Segmentation (IoU tính bằng Pixel) vs. Detection (IoU tính bằng hình hộp chữ nhật).

Gemini Notebook

Viết hàm tính IoU trong Keras

Chuyển lý thuyết thành Tensor operations để chạy song song trên GPU.




```python
from keras import ops

def unpack(box):
    return box[..., 0], box[..., 1], box[..., 2], box[..., 3]

def intersection(box1 , box2):
    cx1 , cy1 , w1 , h1 = unpack(box1)
    cx2 , cy2 , w2 , h2 = unpack(box2)
    left = ops.maximum(cx1 - w1 / 2, cx2 - w2 / 2)
    bottom = ops.maximum(cy1 - h1 / 2, cy2 - h2 / 2)
    right = ops.minimum(cx1 + w1 / 2, cx2 + w2 / 2)
    top = ops.minimum(cy1 + h1 / 2, cy2 + h2 / 2)
    return ops.maximum (0.0, right - left) * ops.maximum (0.0, top - bottom)
```

Loại bỏ vòng lặp operations tối ưu cho GPU.

Sử dụng tensor operations tối ưu cho GPU.

Gemini Notebook

Hàm Hợp nhau (Union)

Tính tổng diện tích 2 hộp và trừ đi phần Giao, sau đó chia để tính tỷ lệ.



```python
def intersection_over_union(box1 , box2):
    cx1 , cy1 , w1 , h1 = unpack(box1)
    cx2 , cy2 , w2 , h2 = unpack(box2)

    # Computes intersection
    intersection_area = intersection(box1 , box2)

    # Computes union
    a1 = ops.maximum(w1, 0.0) * ops.maximum(h1 , 0.0)
    a2 = ops.maximum(w2, 0.0) * ops.maximum(h2 , 0.0)
    union_area = a1 + a2 - intersection_area

    # Divides gracefully , avoiding ZeroDivision
    return ops.divide_no_nan(intersection_area , union_area)
```

Tránh lỗi **ZeroDivision**

Gemini Notebook

Định nghĩa Hàm Mất mát Hộp (YOLO Box Loss)

Đây là tinh hoa của YOLO. Loss sẽ cộng dồn:

Sai số tọa độ trung tâm (xy)

Sai số căn bậc hai chiều cao/rộng (wh)

Sai số Confidence

```python
def signed_sqrt(x):
    return ops.sign(x) * ops.sqrt(ops.absolute(x) + keras.config.epsilon ())

def box_loss(true , pred):
    xy_true , wh_true , conf_true = true[..., :2], true[..., 2:4], true[..., 4:]
    xy_pred , wh_pred , conf_pred = pred[..., :2], pred[..., 2:4], pred[..., 4:]
    no_object = conf_true == 0.0

    xy_error = ops.square(xy_true - xy_pred)
    wh_error = ops.square(signed_sqrt(wh_true) - signed_sqrt(wh_pred))

    iou = intersection_over_union(true , pred)
    conf_target = ops.where(no_object , 0.0, ops.expand_dims(iou , -1))
    conf_error = ops.square(conf_target - conf_pred)
    # Continues on next slide ...
```

Gemini Notebook

Hệ số phạt (Penalty Scaling)



Tác giả Redmon đề xuất các hệ số phạt để cân bằng học tập:

Tăng phạt x5 lần cho lỗi hộp

```
# Concatenates the errors with scaling hacks
error = ops.concatenate(
    (
        ops.where(no_object , 0.0, xy_error * 5.0),
        ops.where(no_object , 0.0, wh_error * 5.0),
        ops.where(no_object , conf_error * 0.5, conf_error),
    ),
    axis=-1,
)
# Returns one loss value per sample; Keras will sum over the batch.
return ops.sum(error , axis=(1, 2, 3))
```

Giảm nhẹ phạt x0.5 cho vùng không có đồ vật (Empty space)

Gemini Notebook

Huấn luyện Mô hình



Cuối cùng, đóng gói thành 1 quá trình biên dịch hoàn chỉnh. Loss của **phân loại** dùng hàm **sparse_categorical_crossentropy** chuẩn.

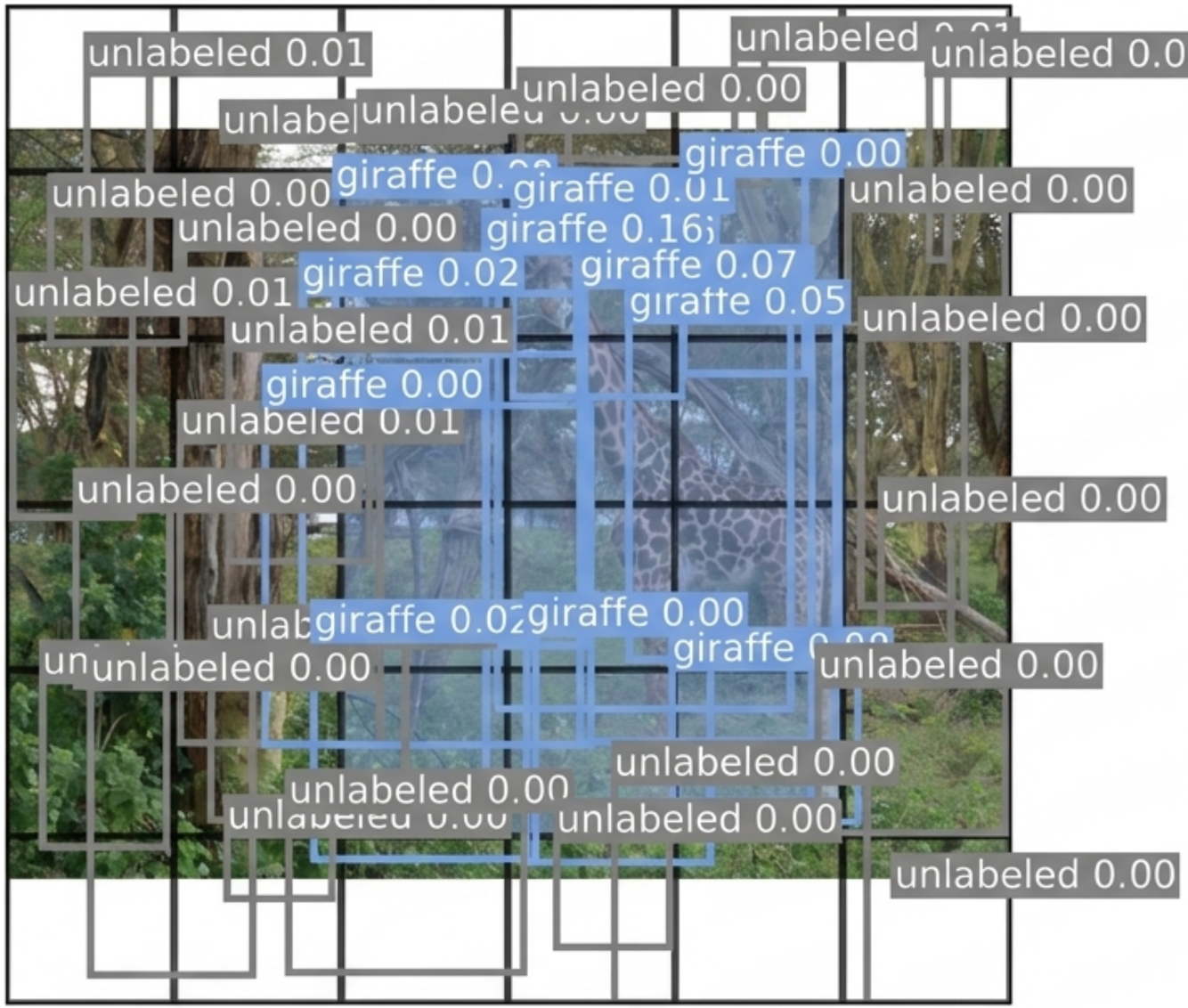
```python
# We use two separate loss functions for the two outputs
model.compile(
    optimizer=keras.optimizers.Adam(2e-4),
    loss={"box": box_loss , "class": "sparse_categorical_crossentropy"},
)

# Start training the detector
model.fit(
    train_dataset ,
    validation_data=val_dataset ,
    epochs=4,
)
```

WARNING **Quá trình có thể tốn hàng giờ trên GPU do khối lượng tính toán quá khổng lồ.**

Gemini Notebook

Kết quả Thô (Raw Predictions)



Hình 12.8: Hàng ngàn hộp dự đoán thô



- **Trong quá trình dự đoán (Inference), MỖI ô lưới sẽ xuất ra nhiều Bounding Boxes.**
- **Mô hình sinh ra hàng trăm hộp giới hạn cùng một lúc chồng chéo lên nhau.**
- **Hầu hết các hộp này là nhiễu, confidence rất thấp.**

Làm sao để dọn dẹp đống hỗn độn này?

Gemini Notebook

Công cụ 2: Non-Maximum Suppression (NMS)

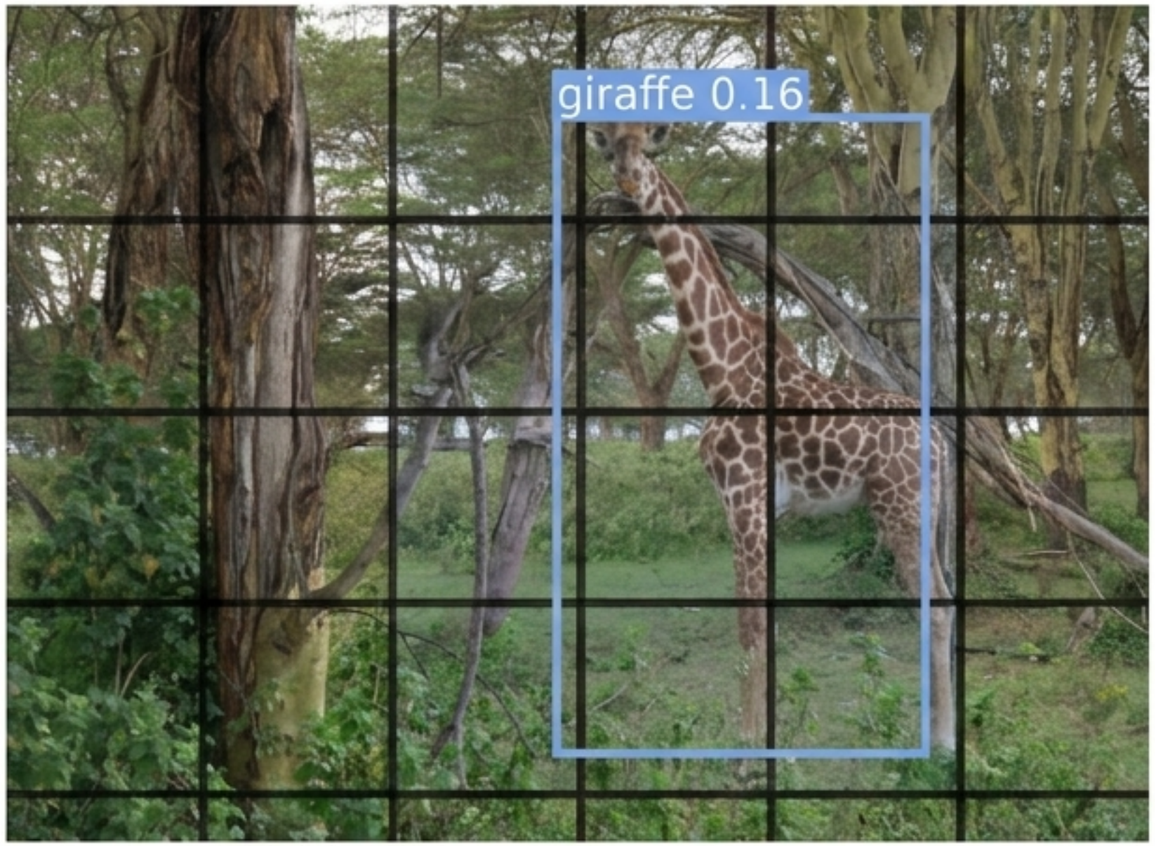


Thuật toán NMS để lọc nhiễu hộp giới hạn:

1. Bỏ tất cả các hộp có Điểm tự tin (Confidence Score) quá thấp.
2. Vòng lặp: Chọn hộp có Confidence Score Cao nhất làm chuẩn (Vd: Hộp chó 0.98).
3. Tính IoU của hộp chuẩn với tất cả các hộp Chó khác còn lại.
4. Xóa toàn bộ các hộp có IoU > 0.4 (Vì chúng chắc chắn là bản sao trùng lặp của hộp chuẩn).
5. Lặp lại đến khi không còn hộp nào chưa xét.

Gemini Notebook

Kết quả Cuối cùng (Sau khi chạy NMS)



Hình 12.9: Kết quả sạch sẽ sau NMS

Sau khi trải qua bộ lọc Non-Maximum Suppression, từ hàng vạn hộp nhiễu, ta chỉ còn lại chính xác số lượng vật thể trong hình.



YOLO vừa nhanh, vừa chính xác, thiết lập tiêu chuẩn công nghiệp (Industry Standard) cho mọi cho mọi ứng dụng Camera AI, Flycam, và Xe tự lái hiện nay.

Gemini Notebook